

Frontend Integration Documentation

Overview

This document describes the frontend integration for the Mindous AI agent platform, enabling real-time visualization of agent execution, code generation, builds, and live previews.

Architecture

Components Hierarchy

```
ChatInterface
├── MessageList
│   ├── UserMessage (Card)
│   └── AgentMessage
│       ├── ActionStream
│       ├── CodeDisplay
│       ├── BuildProgress
│       └── PreviewFrame
```

Core Components

1. Agent Message Component (`/components/agent/agent-message.tsx`)

The unified component for displaying agent execution messages with all interactive elements.

Features:

- Status badges (planning, executing, building, deploying, completed, failed)
- Collapsible details section
- Embedded action stream
- Code artifacts display
- Build progress tracking
- Live preview iframe

Usage:

```
import { AgentMessage } from '@components/agent';

<AgentMessage
  content="Starting execution..."
  agentState={agentExecutionState}
/>
```

2. Action Stream Component (`/components/agent/action-stream.tsx`)

Real-time stream of agent actions with color-coded icons and timestamps.

Action Types:

- `task_started` - Blue
- `task_completed` - Green

- `code_generated` - Purple
- `build_started` - Orange
- `build_completed` - Emerald
- `preview_ready` - Cyan
- `error` - Red

Features:

- Auto-scroll to latest action
- Color-coded by event type
- Timestamps for each action
- Detailed descriptions
- Clickable links for previews

Usage:

```
import { ActionStream } from '@components/agent';

<ActionStream
  actions={agentState.actions}
  autoScroll={true}
/>
```

3. Code Display Component (/components/agent/code-display.tsx)

Syntax-highlighted code viewer with copy functionality.

Features:

- Line numbers
- Syntax highlighting (basic)
- Copy to clipboard
- Collapsible code blocks
- Language indicators
- Multiple language support

Supported Languages:

- TypeScript
- JavaScript
- Python
- CSS
- HTML
- JSON

Usage:

```
import { CodeDisplay } from '@components/agent';

<CodeDisplay
  fileName="app.tsx"
  language="typescript"
  code={generatedCode}
  defaultExpanded={false}
/>
```

4. Build Progress Component (/components/agent/build-progress.tsx)

Visual build progress tracker with logs.

Features:

- Progress bar
- Current build step indicator
- Build duration timer
- Expandable log viewer
- Status indicators (pending, in_progress, completed, failed)
- Error messages display

Usage:

```
import { BuildProgress } from '@components/agent';

<BuildProgress
  buildInfo={{
    buildId: "build-123",
    status: "in_progress",
    currentStep: "Installing dependencies...",
    logs: ["npm install", "Building..."],
    startTime: new Date().toISOString()
  }}
/>
```

5. Preview Frame Component (/components/agent/preview-frame.tsx)

Secure iframe for live app previews with responsive viewport controls.

Features:

- Responsive viewport switcher (desktop, tablet, mobile)
- Refresh button
- Open in new tab
- Loading states
- Error handling
- URL display
- Sandbox security

Viewport Sizes:

- Desktop: 100% width, 600px height
- Tablet: 768px × 1024px
- Mobile: 375px × 667px

Usage:

```
import { PreviewFrame } from '@components/agent';

<PreviewFrame
  previewUrl="http://localhost:3000"
  title="App Preview"
  defaultViewport="desktop"
/>
```

Hooks

useAgentStream Hook (/lib/hooks/useAgentStream.ts)

Custom React hook for subscribing to agent execution streams via Server-Sent Events (SSE).

Features:

- Automatic SSE connection
- Event parsing and state management
- Reconnection on disconnect
- Event callbacks
- Type-safe event handling

Event Types Handled:

- EXECUTION_STARTED
- EXECUTION_PROGRESS
- EXECUTION_COMPLETED
- EXECUTION_FAILED
- STEP_STARTED
- STEP_COMPLETED
- CODE_GENERATION_COMPLETED
- BUILD_STARTED
- BUILD_LOG
- BUILD_COMPLETED
- BUILD_FAILED
- PREVIEW_READY

Usage:

```
import { useAgentStream } from '@lib/hooks/useAgentStream';

const { state, isConnected, error, reconnect } = useAgentStream({
  runId: 'run-123',
  executionId: 'exec-456',
  onAction: (action) => console.log('Action:', action),
  onCodeGenerated: (artifact) => console.log('Code:', artifact),
  onBuildUpdate: (buildInfo) => console.log('Build:', buildInfo),
  onPreviewReady: (url) => console.log('Preview:', url),
  onCompleted: () => console.log('Completed'),
  onError: (err) => console.error('Error:', err),
});

// Access agent state
console.log(state.status); // 'planning' | 'executing' | 'building' | ...
console.log(state.progress); // 0-100
console.log(state.actions); // Array of actions
console.log(state.codeArtifacts); // Array of code files
console.log(state.buildInfo); // Build information
console.log(state.previewUrl); // Preview URL when ready
```

Chat Interface Integration

Enhanced Chat Interface (/components/chat/chat-interface.tsx)

The main chat interface now integrates with the agent execution engine.

Flow:

1. User sends message
2. Call `/api/agent/execute` to start execution
3. Receive `executionId` and `runId`
4. Create agent message placeholder
5. Subscribe to SSE stream via `useAgentStream`
6. Update agent message in real-time with state changes
7. Display actions, code, builds, and previews as they occur

Key Changes:

- Added `activeExecution` state to track current execution
- Integrated `useAgentStream` hook
- Real-time state updates via `useEffect`
- Disabled input during execution
- Status indicator in header

Message Types

Updated Message Interface

```
export interface Message {
  id: string;
  role: 'user' | 'assistant' | 'system' | 'agent';
  content: string;
  createdAt: Date;
  agentState?: AgentExecutionState;
}
```

Agent Execution State

```
export interface AgentExecutionState {
  runId: string;
  executionId: string;
  status: 'idle' | 'planning' | 'executing' | 'building' | 'deploying' | 'completed' | 'failed';
  currentStep?: string;
  progress: number;
  totalSteps: number;
  completedSteps: number;
  actions: AgentAction[];
  codeArtifacts: CodeArtifact[];
  buildInfo?: BuildInfo;
  previewUrl?: string;
  error?: string;
}
```

API Integration

Agent Execution Endpoint

POST `/api/agent/execute`

Request:

```
{
  "prompt": "Create a todo app with Next.js",
  "context": {
    "taskType": "code",
    "complexity": "medium",
    "constraints": ["Use TypeScript", "Use Tailwind CSS"]
  }
}
```

Response:

```
{
  "success": true,
  "executionId": "exec-uuid",
  "runId": "run-uuid",
  "message": "Agent execution started"
}
```

SSE Stream Endpoint

GET /api/streams/runs/[runId]

Query Parameters:

- lastEventId (optional): Resume from specific event

Event Format:

```
event: event
data: {
  "id": "event-uuid",
  "runId": "run-uuid",
  "subtaskId": "subtask-uuid",
  "eventType": "CODE_GENERATION_COMPLETED",
  "message": "Code generated",
  "data": { ... },
  "timestamp": "2024-01-01T12:00:00Z"
}
```

Styling

All components use:

- Tailwind CSS for styling
- shadcn/ui components as base
- Consistent color scheme:
 - Blue: Planning/In Progress
 - Green: Success/Completed
 - Red: Error/Failed
 - Orange: Building
 - Purple: Code Generation
 - Cyan: Preview

Testing

Manual Testing Flow

1. Start the development server:

```
bash
```

```
npm run dev
```

2. Navigate to chat interface

3. Send a message: "Create a simple todo app"

4. Verify:

- ☒ User message appears
- ☒ Agent message placeholder created
- ☒ Status indicator shows in header
- ☒ Action stream updates in real-time
- ☒ Code artifacts display when generated
- ☒ Build progress shows when building
- ☒ Preview iframe loads when ready
- ☒ All components are interactive

Component Testing

Test individual components in isolation:

```
// Test ActionStream
const testActions = [
  {
    id: '1',
    type: 'task_started',
    title: 'Planning execution',
    timestamp: new Date().toISOString(),
  },
  // ... more actions
];

<ActionStream actions={testActions} />
```

Best Practices

1. Error Handling

- Always handle SSE connection errors
- Show user-friendly error messages
- Provide retry mechanisms

2. Performance

- Use React.memo for heavy components
- Implement virtual scrolling for long logs
- Debounce rapid state updates

3. Accessibility

- Provide keyboard navigation
- Add ARIA labels
- Ensure sufficient color contrast

4. Security

- Sandbox iframes with appropriate permissions
- Validate preview URLs
- Sanitize user input

Troubleshooting

SSE Connection Issues

Problem: Stream not connecting

Solution:

- Check if run exists and belongs to user
- Verify Redis is running
- Check browser console for errors

Preview Not Loading

Problem: Iframe shows error

Solution:

- Verify preview URL is accessible
- Check sandbox permissions
- Ensure CORS headers are correct

Code Not Displaying

Problem: Code artifacts not showing

Solution:

- Verify `CODE_GENERATION_COMPLETED` event is fired
- Check code content is not empty
- Inspect `agentState` in React DevTools

Future Enhancements

1. Syntax Highlighting

- Integrate Prism.js or highlight.js
- Support more languages

2. Code Editing

- Allow inline code editing
- Request modifications via agent

3. Preview Interactions

- Enable testing interactions in preview
- Record user flows

4. Collaboration

- Multi-user agent executions
- Shared preview sessions

5. History

- View past executions
- Replay agent actions

Resources

- [SSE Specification](https://html.spec.whatwg.org/multipage/server-sent-events.html) (https://html.spec.whatwg.org/multipage/server-sent-events.html)
- [shadcn/ui Components](https://ui.shadcn.com/) (https://ui.shadcn.com/)
- [Tailwind CSS](https://tailwindcss.com/) (https://tailwindcss.com/)
- [Next.js Documentation](https://nextjs.org/docs) (https://nextjs.org/docs)